这篇的目标就一个 `OP_CALL` 函数调用，虽然只有一个指令，但是这个指令涉及的东西很多，要从头理解语言之间函数的调用

```
// A B C  R[A], ... ,R[A+C-2] := R[A](R[A+1], ... ,R[A+B-1])
L_OP_CALL: {
    StkId ra = RA(i);
    CallInfo *newci; // 新建一个调用栈
    int b = GETARG_B(i);
    int nresults = GETARG_C(i) - 1; // 返回值数量，比如 a,b = foo(1,2,3) 这种情况
    if (b != 0) /* B不等0的时候说明解释器知道参数数量是多少 */
        L->top.p = ra + b; /* 设置栈顶 */
    /* else previous instruction set top */
    // 变长参数调用，参数数量运行期才知道，top 已经被设置，OP_CALL 不改
    savepc(ci); /* in case of errors 存储起来当前pc */
    if ((newci = luaD_precall(L, ra, nresults)) == NULL) // 进行调用
        updatetrap(ci); /* C call; nothing else to be done */
    else { /* Lua call: run function in this same C frame */
        ci = newci;
        goto startfunc;
    }
    vmbreak;
}
```

先来看注释

- 从寄存器内拿到 A 作为参数
- 括号内的加数代表参数 B 代表参数数量(这个在语言语法解析时候决定的，现在只是负责使用)

```
static void funcargs(LexState *ls, expdesc *f)
{
    FuncState *fs = ls->fs;
    expdesc args;
    int base, nparams;
    int line = ls->linenumber;
```

```
    switch (ls->t.token) {
    case '(': { /* funcargs -> '(' [ explist ] ')' */
        luaX_next(ls);
        if (ls->t.token == ')') /* arg list is empty? */
            args.k = VVOID;
        else {
            explist(ls, &args);
            if (hasmultret(args.k))
                luaK_setmultret(fs, &args);
        }
        check_match(ls, ')', '(', line);
        break;
    }
    case '{' /*}*/: { /* funcargs -> constructor */
        constructor(ls, &args);
        break;
    }
    case TK_STRING: { /* funcargs -> STRING */
        codestring(&args, ls->t.seminfo.ts);
        luaX_next(ls); /* must use 'seminfo' before 'next' */
        break;
    }
    default: {
        luaX_syntaxerror(ls, "function arguments expected");
    }
    }
    lua_assert(f->k == VNONRELOC);
    base = f->u.info; /* base register for call */
    if (hasmultret(args.k))
        nparams = LUA_MULTRET; /* open call */
    else {
        if (args.k != VVOID)
            luaK_exp2nextreg(fs, &args); /* close last argument */
        nparams = fs->freereg - (base + 1);
    }
    init_exp(f, VCALL, luaK_codeABC(fs, OP_CALL, base, nparams + 1, 2)); // 参数数量加一
    luaK_fixline(fs, line);
    /* call removes function and arguments and leaves one result (unless
      changed later) */
    fs->freereg = cast_byte(base + 1);
}
```

- B 在编译期写入 top 的位置，参数数量是 top - (ra+1)，所以 VM 里总是用 B-1 来得到实际参数数量
- `B` 和 `C` 的编码都是 **为了和寄存器布局对齐**，VM 的操作都是“偏移量 = base + B 或 C”

  我们来向下看调用的代码

```
CallInfo *luaD_precall(lua_State *L, StkId func, int nresults)
{
    unsigned status = cast_uint(nresults + 1);
    lua_assert(status <= MAXRESULTS + 1);
retry:
    switch (ttypetag(s2v(func))) {
    case LUA_VCCL: /* C closure */
        precallC(L, func, status, clCvalue(s2v(func))->f);
        return NULL;
    case LUA_VLCF: /* light C function */
        precallC(L, func, status, fvalue(s2v(func)));
        return NULL;
    case LUA_VLCL: { /* Lua function */
         CallInfo *ci;
         Proto *p = clLvalue(s2v(func))->p; // 获取Proto（字节码等）
         int narg = cast_int(L->top.p - func) - 1; // 实际参数数
         int nfixparams = p->numparams; // 定义参数个数
         int fsize = p->maxstacksize;   // 帧大小
         checkstackp(L, fsize, func);
         L->ci = ci = prepCallInfo(L, func, status, func + 1 + fsize);
         ci->u.l.savedpc = p->code; /* 开始执行的位置 */
         for (; narg < nfixparams; narg++)
             setnilvalue(s2v(L->top.p++)); /* 补齐缺少参数 */
         return ci;
    }
    default: { /* not a function */
        checkstackp(L, 1, func); /* space for metamethod */
        status = tryfuncTM(L, func, status); /* try '__call' metamethod */
        goto retry; /* try again with metamethod */
    }
    }
}
// 参数传进来本身状态，函数本身的调用信息，调用状态标记，比如返回值数量和 `CIST_C` 标志
```

```c
// 要调用的 C 函数指针 (int (*)(lua_State*))
l_sinline int precallC(lua_State *L, StkId func, unsigned status,
lua_CFunction f)
{
    int n; /* number of returns */
    CallInfo *ci;
    checkstackp(L, LUA_MINSTACK, func); /* Lua 对栈有严格要求，这里确保栈至少能
容纳 LUA_MINSTACK 个元素 */
    L->ci = ci = prepCallInfo(L, func, status | CIST_C, L->top.p +
LUA_MINSTACK); // 创建新的栈帧,并把CIST_C标志标记在状态里
    // Hook机制允许调试器或 profiler 捕获函数调用事件，不影响正常运行逻辑
    if (l_unlikely(L->hookmask & LUA_MASKCALL)) {
        int narg = cast_int(L->top.p - func) - 1;
        luaD_hook(L, LUA_HOOKCALL, -1, 1, narg);
    }
    lua_unlock(L); // 允许 C 函数执行 Lua API 调用
    n = (*f)(L); /* do the actual call */
    lua_lock(L); // 函数返回后重新锁定 Lua 状态
    api_checknelems(L, n); // Lua API 检查 C 函数返回值数量是否合法
    luaD_poscall(L, ci, n); // 完成 C 函数调用后的栈清理
    return n;
}
```

然后Lua函数会走到else分支内，将当前ci更改为新调用信息，然后开始递归调用代码